BDH → AttnGrounder

A Mechanistic Study Guide: What We Built and the Math Behind It

Sudarshan Sudhakar

July 30, 2026

One-sentence pitch

AttnGrounder originally grounds a phrase by having every image region compute a softmax-weighted average over the command's word vectors. We replaced that one attention module with BDH's mechanism — lift regions and words into a high-dimensional sparse space, build one explicit associative-memory matrix from the words via an outer product, let every region read from that same memory, and combine what a region already represents with what it retrieved *multiplicatively* rather than by averaging — to test whether this sharper, memory-based readout disambiguates scenes with multiple same-class objects better than softmax does.

Contents

1 The Baseline: AttnGrounder's Original Attention

File: `external/AttnGrounder/model/grounding_model.py`, method `text_attn` (∼line 199).

Inputs / outputs.

- `image_feat`: (B, C, H, W) — a grid of visual region features, one C-dim vector per spatial cell ($C = 512$).
- `lang_feat`: (B, T, C) — one feature vector per word, from a BiLSTM over GloVe embeddings.
- Output 1, `lang_feat_attn`: (B, C, H, W) — for every region, "what does the text say about this region," same shape as `image_feat` so it can be concatenated back with it.
- Output 2, `beta`: (B, H, W) — a heatmap in $[0, 1]$, one scalar per region, supervised directly by a BCE loss against the ground-truth box mask (an auxiliary loss forcing the attention to light up on the right region).

The math. Let $R = H \cdot W$ be the number of regions and T the number of words. With $G \in \mathbb{R}^{R \times C}$ the flattened region features and $Q \in \mathbb{R}^{T \times C}$ the word features:

$$S = G\,Q^\top \qquad \in \mathbb{R}^{R \times T} \quad \text{(raw region–word dot products)} \tag{1}$$

$$A = \mathrm{softmax}_T(S) \qquad \in \mathbb{R}^{R \times T} \quad \text{(normalize over words, per region)} \tag{2}$$

$$\text{lang\_feat\_attn} = A\,Q \qquad \in \mathbb{R}^{R \times C} \quad \text{(weighted average of word vectors)} \tag{3}$$

$$\beta = \sigma\Big(\textstyle\sum_t S_{\cdot,t}\Big) \qquad \in \mathbb{R}^{R} \quad \text{("how strongly does this region match } \textit{any} \text{ word")} \tag{4}$$

This is standard **cross-attention**: every region asks "which words describe me?", gets a softmax distribution over the T words, and takes a weighted average of their embeddings. It has **zero learned parameters** — purely dot-product → softmax → sigmoid on features already produced upstream by the CNN and BiLSTM.

Why this is a weak spot

Softmax attention gives every region a smooth, *convex combination* of word vectors. If the phrase is "the car on the left" and there are two cars in the scene, both car-regions score similarly against the word "car" — softmax just averages; it has no mechanism to sharply separate "this car" from "that car" beyond whatever the raw upstream features already encode. There is no notion of a persistent, content-addressable memory — it is a single, one-shot, low-capacity read.

2 What BDH Is: Intuition, Then Math

BDH = "Baby Dragon Hatchling" (ground-truth code: `external/bdh/bdh.py`; paper `papers/2509.26507v1.md`). Mechanically, forget the biological framing for defense purposes: it is a **linear-attention Transformer with an explicit outer-product associative memory**, built from three ideas.

2.1 Lift into a big, sparse, positive space

Instead of attending directly in the model's working dimension d (e.g. 512), BDH first **lifts** every feature vector into a much higher dimension $n = \texttt{mult} \times d$ (we use $\texttt{mult} = 4 \Rightarrow n = 2048$) via a learned linear map $E \in \mathbb{R}^{d \times n}$, followed by ReLU:

$$x_{\text{sparse}} = \mathrm{ReLU}(x\,E) \in \mathbb{R}^n, \qquad n \gg d. \tag{5}$$

Intuition

In a low dimension d, two similar-but-different concepts (car #1, car #2) sit close together and small distinguishing signals get lost. Projecting into a much higher dimension and clipping negatives with ReLU is exactly what sparse coding / kernel feature maps do: it is far easier to make two points *linearly separable* and *sparse* (few active neurons each) in a big space than a small one. Only a handful of the n neurons fire for any given input, and different inputs tend to activate mostly non-overlapping subsets of neurons — like a hash. (BDH's paper motivates this via a Hebbian/neuronal analogy — "neurons that fire together wire together" — but functionally it is a sparse, positive, overcomplete ReLU feature map, in the same family as ReLU kernel feature maps used in linear attention.)

2.2 Attention becomes an explicit associative memory

Standard attention computes $\text{softmax}(QK^\top)V$: normalize scores over keys, then average values. That is a *soft lookup table with no persistent state* — every query re-derives its weights from scratch against all keys.

BDH instead builds a memory matrix explicitly. With $K_{\text{sparse}}, Q_{\text{sparse}} \in \mathbb{R}^{\cdot\times n}$ the lifted keys/queries and $V \in \mathbb{R}^{\cdot\times d}$ the values:

$$\rho = K_{\text{sparse}}^\top V \qquad \in \mathbb{R}^{n\times d} \quad \text{(outer-product accumulation over } \textit{all} \text{ keys/values)} \tag{6}$$

$$\text{read} = Q_{\text{sparse}}\,\rho \qquad \in \mathbb{R}^{\cdot\times d} \quad \text{(each query retrieves by dotting into that memory)} \tag{7}$$

This is **linear attention**: no softmax, just a weighted sum of outer products $\sum_i k_i \otimes v_i$, then each query multiplies against that sum. Algebraically it is equivalent to $Q_{\text{sparse}}K_{\text{sparse}}^\top V$ (re-associated for efficiency) — i.e. "unnormalized linear attention." What matters for the grounding use case is the **framing**: ρ is a literal memory matrix that can be *built once* (accumulate keys and values into it) and *read many times* (any query dots into it) — content-addressable storage, like a Hopfield network or a linear key–value store, rather than a fresh softmax computed per query.

Why this maps well onto grounding

In our use, the "keys/values" are the words of the command and the "queries" are image regions. Building the memory once from all words, then letting every region address into it, is literally what a *referring expression* should behave like: a small declarative store ("car", "left", "white") that every candidate region queries against.

2.3 A multiplicative gate, not a linear blend

After the memory read, BDH does not simply decode `read` back to d dimensions. It re-lifts the read result into the same sparse n-dim space and **multiplies it, element-wise**, against the original query's own sparse address:

$$\text{read}_{\text{sparse}} = \text{ReLU}(\text{read}\,E_v) \qquad \in \mathbb{R}^{\cdot\times n} \tag{8}$$

$$\text{gated} = Q_{\text{sparse}} \odot \text{read}_{\text{sparse}} \qquad \in \mathbb{R}^{\cdot\times n} \quad \text{(elementwise product)} \tag{9}$$

$$\text{out} = \text{gated}\,D \qquad \in \mathbb{R}^{\cdot\times d} \quad \text{(decode)} \tag{10}$$

Why multiplicative matters

A weighted *sum* (softmax attention) can only ever interpolate between existing value vectors — it is linear in the values. A gated *product* between "what I was looking for" (Q_{sparse}) and "what I found" ($\text{read}_{\text{sparse}}$) creates a **bilinear** interaction: the output carries signal only where a neuron is active in *both* the query's own representation and the retrieved memory. This is a sharper, AND-like combination (vs. softmax's OR-like/average combination) and is the mechanism most directly responsible for BDH's claimed ability to keep near-duplicate concepts (e.g. two same-class objects) separable rather than blurred together.

The whole engine, in one line: *lift → build memory (outer product) → read → gate multiplicatively → decode.* Every layer of the original BDH language model (`external/bdh/bdh.py`, class `BDH`) repeats this, plus RoPE positional encoding and a causal mask (it is a next-token predictor). Our job was to strip the parts that only make sense for autoregressive text and repurpose the memory mechanism for *cross-modal grounding*, which has no sequence order between "regions" and "words" at all.

3 What We Built: `bdh_grounding/bdh_fusion.py`

We wrote `BDHVisualTextAttention`, a **drop-in replacement** for `text_attn` — identical inputs/outputs — selected in `grounding_model.py` via `variant == "bdh"` (`self.bdh_attn = BDHVisualTextAttention(...)`, one instance shared across AttnGrounder's 3 FPN scales).

Three variants exist in code (the `mode` flag), but **only mode "C" is validated and used for reported results** — A and B are ablations kept for future comparison.

3.1 Mode C (primary, validated): symmetric, non-causal

This is the direct translation of Section 2 into cross-attention. Let $G \in \mathbb{R}^{R\times d}$ be the region features and $Q \in \mathbb{R}^{T\times d}$ the word features (both LayerNorm'd first):

$$G_q = \text{ReLU}(G\,E) \qquad \in \mathbb{R}^{R\times n} \quad \text{region } \textit{query} \text{ addresses} \tag{11}$$

$$K_k = \text{ReLU}(Q\,E) \qquad \in \mathbb{R}^{T\times n} \quad \text{word } \textit{key} \text{ addresses} \tag{12}$$

$$\rho = K_k^{\top} Q \qquad \in \mathbb{R}^{n\times d} \quad \text{memory, built once from } \textit{all } T \text{ words} \tag{13}$$

$$\text{read} = (G_q\,\rho)\cdot \tfrac{1}{\sqrt{n}} \qquad \in \mathbb{R}^{R\times d} \quad \text{each region reads} \tag{14}$$

$$\text{read}_{\text{sparse}} = \text{ReLU}\left(\text{LN}(\text{read})\,E_v\right) \tag{15}$$

$$\text{gated} = G_q \odot \text{read}_{\text{sparse}} \qquad \in \mathbb{R}^{R\times n} \quad \text{multiplicative gate} \tag{16}$$

$$T' = \text{LN}(\text{gated}\,D) \qquad \in \mathbb{R}^{R\times d} \;\to\; (B,C,H,W) \tag{17}$$

$$\beta = \sigma\big(\text{Linear}_{d\to 1}(\text{read})\big) \qquad \in \mathbb{R}^{R} \;\to\; (B,H,W) \tag{18}$$

Listing 1: Mode C forward pass (`bdh_grounding/bdh_fusion.py:_forward_C`), abbreviated

```
Gq  = relu(G @ E)              # (B, R, n) region query addresses
Kk  = relu(Q @ E)              # (B, T, n) word key addresses
rho = Kk.transpose(1, 2) @ Q   # (B, n, d) associative memory, built from ALL words
read = (Gq @ rho) * scale      # (B, R, d) each region reads from memory
read_sparse = relu(ln(read) @ Ev)
gated = Gq * read_sparse       # (B, R, n) multiplicative gate
Tprime = ln(gated @ Dx)        # (B, R, d) -> reshaped to (B, C, H, W)
beta = sigmoid(beta_head(read)) # learned scalar head -> (B, H, W)
```

Mapping onto Section 2/3.

- **Regions are queries, words are keys/values.** This is the *opposite* direction from BDH's original language-model use (where a token queries *past* tokens), but here there is no "past": grounding is symmetric / non-causal, so we drop BDH's causal mask and RoPE entirely — word order within the sentence doesn't gate what a region is allowed to attend to.
- $\rho = K^\top Q$ is built **once per forward pass** from all T words — literally "memorize the whole command" — then every one of the $R = H \times W$ regions reads from that *same* memory independently. This replaces the baseline's per-region softmax-over-words with per-region dot-product-into-memory.
- The gate $G_q \odot \text{read}_{\text{sparse}}$ is the bilinear AND from Section 2.3: a region's own sparse signature must overlap with what its memory read up in order to produce a large output, rather than the baseline's linear softmax average.

[
Why did β need a learned head?] Unlike the baseline's parameter-free $\sigma(\sum S)$, mode C uses `sigmoid(Linear(`$d \to 1$`)(read))`. BDH's ReLU-sparse dot products (G_q, K_k) are all ≥ 0 by construction, so a raw sum of them can never dip meaningfully below the sigmoid's midpoint. The auxiliary BCE mask loss needs $\beta \to 0$ on background regions — unreachable without a learned bias/weight to shift it. This is a $\sim (d{+}1)$-parameter head, negligible in size but necessary: BDH is *not* fully parameter-free here, by design, for exactly this reason.

3.2 Fusion trim (parameter budget)

Baseline AttnGrounder concatenates three streams before its output convolution: [visual, lang_feat_attn, $\beta\odot$visual], i.e. `embin_size` = `emb_size`$\times 3$. For the BDH variant we fuse only [visual, lang_feat_attn] (drop the $\beta\odot$visual product stream): `embin_size` = `emb_size` $\times\, 2$. This is a parameter-budget decision: it guarantees the BDH-variant model's total parameter count does not exceed the 75.84M baseline, so any accuracy difference is not confounded by "we just gave the model more capacity." `mult` $= 2$ is the default operating point, balancing BDH's need for $n \gg d$ (expansion gives sparse separability, but costs $n{\times}d$ params per matrix) against that budget.

3.3 Modes A and B: implemented ablations

Mode A

Text is first passed through a *causal* BDH self-attention pass over itself (words attend only to earlier words), producing a contextualized memory, and *then* regions query that. Tests whether giving the text module its own BDH "understanding pass" before grounding helps — an in-context-retrieval framing closer to BDH's original LM use.

Mode B

The most literal port of the original `bdh.py` forward pass: concatenate [words ; regions] into *one* sequence, run genuine causal BDH self-attention (with RoPE) over the whole thing, then read out only the region positions at the end — "vanilla BDH" with regions appended as extra, causally-later tokens.

Mode C was chosen as primary because grounding has no real notion of sequence order between "words" and "regions": forcing a causal mask (A, B) throws away information for no modeling reason, whereas C's symmetric non-causal memory matches the task structure directly.

3.4 Cross-scale memory ("growing" prior)

AttnGrounder runs `text_attn` **independently at 3 FPN scales** (different $H \times W$ resolutions). We added an optional `region_prior` argument: each scale's own `lang_feat_attn` output can be fed forward (upsampled + added as a residual) into the next, finer scale's region features before that scale builds its own memory — the "growing memory across scales" idea, orthogonal to the A/B/C mode choice. Off by default (`region_prior=None` reproduces the original per-scale-independent behavior exactly), toggled via `bdh_growing_scales`.

4 Side-by-Side: What Actually Changed, Mathematically

	Baseline (softmax cross-attn)	**BDH (mode C)**
Score function	$QK^\top$ (dot product in raw d-dim space)	$Q_{\text{sparse}}(K_{\text{sparse}}^\top V)$ — score pre-aggregated into a memory matrix
Normalization	softmax over words (sums to 1)	none — unnormalized linear attention, scaled by $1/\sqrt{n}$
Combine rule	weighted **sum** of value vectors (linear, convex)	**outer-product memory read**, then elementwise-**multiplied** with the query's own sparse code (bilinear)
Feature space	raw d-dim (e.g. 512)	lifted n-dim sparse ReLU space ($n =$ `mult` $\cdot d$, e.g. 2048), $n \gg d$
Params	0	E, E_v, D ($\approx 3nd$) + tiny β head
Complexity	$O(RT\,d)$	$O(Rn) + O(Tn) + O(nd)$ — memory ρ built once, reused by every region

The one sentence version

Softmax attention computes a fresh weighted average per region; BDH compiles the entire command into one small memory matrix once, then lets every region address into it through a sparse, multiplicative (AND-like) readout instead of a smooth average.

5 The Hypothesis: Why This Should Help Ambiguous Scenes

This is the hypothesis actually being tested. Same-class ambiguity (two cars in frame, command says "the car on the left") is exactly where **softmax averaging degrades**: if both car-regions score similarly against the word "car", softmax spreads a similar-shaped distribution across both, and the linear weighted-sum blends their word evidence almost identically.

BDH's sparse lift makes it more likely that subtly different regional features (car-on-left vs. car-on-right, differing e.g. in local context) activate **different subsets** of the n sparse neurons, and the multiplicative gate then produces sharply different outputs for regions whose sparse codes do not overlap the retrieved memory the same way — i.e. BDH has structurally more room to *disambiguate* than a low-dimensional softmax does.

This is why evaluation is **stratified**: `analysis/stratified_eval.py` splits AP50 into "ambiguous" (≥ 1 other same-class object in the scene, from nuScenes metadata) vs. "unambiguous," specifically to check whether any BDH gain *concentrates* where the hypothesis predicts it should, rather than being a uniform, unexplained bump.

6 Anticipated Questions

[
Is this the actual BDH paper's architecture?] No — it is BDH's *core mechanism* (sparse lift + outer-product memory + multiplicative gate) repurposed as a cross-attention module. We dropped what only applies to autoregressive language modeling: the causal mask, RoPE positional encoding, and the token embedding / LM head. Mode B is the closest to "literal BDH"; mode C is the adapted version and the one we report.

[
Isn't $\rho = K^\top Q$ just a matmul reordering trick?] Algebraically $Q(K^\top V) = (QK^\top)V$, so yes — it is the identity linear attention always uses. The "memory" framing matters because ρ is a fixed (n, d) object built *once* per forward pass and reused by every one of the R region queries — it does not recompute anything per-region, unlike softmax's $QK^\top$, which is inherently a full $R \times T$ pairwise matrix. It is a legitimate architectural distinction (persistent, content-addressable state vs. per-query recomputation), not just terminology.

[
What's the parameter budget vs. the baseline?] Kept $\leq$ 75.84M via the fusion trim (Section 3.2); `mult` and `share_qv_encoder` are the knobs that trade memory capacity (n) against parameter count. `share_qv_encoder=true` ties the query-address and value-path encoders ($E = E_v$), reclaiming $n \times d$ parameters.

[
Did you validate modes A and B too?] Implemented, not the headline result — kept as future ablation work; only mode C has been trained and evaluated end-to-end.

[
What about TransVG?] A second, independent integration (`bdh_grounding/bdh_selfattn.py`) swaps BDH's mechanism into TransVG's transformer *self*-attention (not cross-attention) — same lift→memory→gate recipe, but built signature-compatible with `nn.MultiheadAttention` so it drops straight into TransVG's encoder layers, with padded tokens masked out of the memory (ρ) so they never contribute. This was the cross-architecture validation experiment: does the effect show up in a second, unrelated grounding architecture, not just AttnGrounder.

7 Summary Statement

"AttnGrounder originally grounds a phrase by having every image region do a softmax-weighted average over the command's word vectors — a smooth, parameter-free lookup. We replaced that one module with BDH's mechanism: lift both regions and words into a much higher-dimensional sparse space, build one explicit associative-memory matrix from all the words via an outer product, let every region read from that same memory, then combine the region's own sparse code with what it retrieved *multiplicatively* rather than by averaging. It's mathematically linear attention with a sparse ReLU feature map and a bilinear gate instead of softmax's convex combination, and the hypothesis is that this sharper, AND-like combination rule should specifically help disambiguate scenes with multiple same-class objects — which is why we stratify evaluation by scene ambiguity rather than just reporting one aggregate AP50."

Where to look yourself, to re-derive any line:

- Baseline attention: `external/AttnGrounder/model/grounding_model.py:199`
- BDH ground truth: `external/bdh/bdh.py` (class `BDH`, `forward`)
- AttnGrounder integration: `bdh_grounding/bdh_fusion.py` (`_forward_C` is the one that matters)
- TransVG integration: `bdh_grounding/bdh_selfattn.py`
- Design rationale: project memory notes on the locked module design and evaluation plan